

FULL STACK DEVELOPMENT

FASE 2

AULLA 3 - GRPC COM NODEJS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	10
REFERÊNCIAS.....	11

END

O QUE VEM POR AÍ?

Nesta aula, aprenderemos o básico sobre gRPC, um framework desenvolvido pelo Google. Exploraremos como o gRPC pode ser utilizado para comunicação eficiente entre serviços distribuídos, aproveitando tecnologias modernas como HTTP/2 e Protocol Buffers (protobuf).



HANDS ON

Nesta aula prática, demonstraremos como trabalhar com o gRPC, utilizando o mesmo sistema desenvolvido na aula anterior para uma empresa de investimentos que utilizava REST. O objetivo é mostrar como transformar a comunicação baseada em REST para gRPC.

EXEMPLO

SAIBA MAIS

O gRPC foi criado pelo Google com o objetivo de oferecer um serviço de alto desempenho para chamadas de procedimentos remotos (RPC, Remote Procedure Call em inglês e Chamada de Procedimento Remoto em português).

Ele é uma tecnologia open source que pode ser utilizada em qualquer ambiente, independentemente da linguagem de programação ou plataforma. Além disso, o gRPC suporta balanceamento de carga, rastreamento, verificação de integridade e autenticação, tornando-o uma solução robusta para comunicação entre serviços distribuídos.

Mas antes de avançarmos, o que seria o RPC (Remote Procedure Call)?

Ele é um protocolo que permite a execução de código em uma máquina remota de forma semelhante a uma chamada de função local. Em outras palavras, uma aplicação pode chamar um procedimento (função ou sub-rotina) localizado em outro endereço de rede, e o RPC se encarrega de transmitir os parâmetros, executar o procedimento no servidor remoto e retornar o resultado para o cliente.

Como funciona o RPC?

1. Cliente inicia a chamada: o cliente faz uma chamada de função, como faria normalmente.
2. Marshalling dos parâmetros: os parâmetros da função são serializados (marshalling) para um formato que pode ser transmitido pela rede.
3. Transmissão pela rede: os parâmetros serializados são enviados pela rede para o servidor.
4. Deserialization no servidor: no servidor, os parâmetros são deserializados (unmarshalling).
5. Execução da função: o servidor executa a função com os parâmetros recebidos.
6. Retorno dos resultados: o resultado da execução é serializado e enviado de volta ao cliente.
7. Deserialization no cliente: o cliente deserializa o resultado e continua a execução normalmente.

Vantagens do RPC

O RPC simplifica a comunicação em rede ao abstrair a complexidade de envio e recebimento de mensagens. Além disso, permite que funções remotas sejam chamadas de forma transparente, como se fossem funções locais, e facilita a reutilização de serviços e funcionalidades distribuídas.

Desafios do RPC

No RPC, as chamadas remotas geralmente têm maior latência devido à comunicação pela rede, a comunicação remota pode falhar devido a problemas de rede, exigindo tratamento de erro robusto, e podem haver desafios de compatibilidade entre diferentes sistemas e linguagens de programação.

Para compreender como o gRPC opera e como ele otimiza a comunicação entre serviços, é essencial entender o conceito de RPC (Remote Procedure Call).

O gRPC é uma implementação moderna e otimizada de RPC, utilizando tecnologias avançadas como HTTP/2 e Protocol Buffers para melhorar a performance e a eficiência na comunicação entre serviços distribuídos.

Conforme mencionado, o gRPC trabalha com Protobuf (Protocol Buffers), um método desenvolvido pelo Google para serializar dados estruturados. Essa tecnologia torna a comunicação entre serviços mais eficiente ao oferecer uma forma simples e agnóstica em relação à linguagem de definir estruturas de dados, semelhante ao XML, mas significativamente mais rápida e simples.

Caso você esteja tendo o seu primeiro contato com Protocol Buffers (protobuf) nesta aula, eles são utilizados para definir a estrutura das mensagens trocadas entre o cliente e o servidor no gRPC. Eles permitem que os dados sejam compactados e transmitidos de forma eficiente, melhorando a velocidade e a performance da comunicação.

Vejamos um exemplo de como definir um serviço e as mensagens utilizando protobuf:

```
//nome do arquivo helloworld.proto  
  
syntax = "proto3";  
  
package helloworld;
```

```
message HelloRequest {  
  string name = 1;  
}  
  
message HelloReply {  
  string message = 1;  
}  
  
service Greeter {  
  rpc SayHello (HelloRequest) returns (HelloReply);  
}
```

Neste exemplo, definimos:

- **syntax** = "proto3": define que estamos usando a versão 3 da sintaxe protobuf.
- **package helloworld**: define o pacote para evitar conflitos de nomes.
- **message HelloRequest**: define uma mensagem com um campo name do tipo string.
- **message HelloReply**: define uma mensagem com um campo message do tipo string.
- **service Greeter**: define um serviço chamado Greeter com um método SayHello que recebe uma HelloRequest e retorna uma HelloReply.

Após definir o arquivo .proto, o próximo passo é usar o compilador protobuf (protoc) para gerar classes de cliente e servidor nas linguagens de programação desejadas. Essas classes geradas facilitam a implementação dos serviços gRPC e a comunicação entre o cliente e o servidor.

Para fixarmos o conceito, vamos criar um projeto Node.js para que vejamos este fluxo rodando.

```
mkdir (crie um diretorio para o seu projeto)  
cd (seu projeto)  
npm init -y (para inicializar o node)  
npm install @grpc/grpc-js @grpc/proto-loader (pacotes que vamos utilizar)
```

Crie um novo arquivo chamado server.js e outro chamado client.js. Em seguida, atualize-os conforme os passos abaixo:

Server.js

```
const grpc = require('@grpc/grpc-js');
const protoLoader = require('@grpc/proto-loader');

const packageDefinition = protoLoader.loadSync('helloworld.proto', {
  keepCase: true,
  longs: String,
  enums: String,
  defaults: true,
  oneofs: true
});
const helloProto = grpc.loadPackageDefinition(packageDefinition).helloworld;

function sayHello(call, callback) {
  callback(null, { message: 'Hello ' + call.request.name });
}

function main() {
  const server = new grpc.Server();
  server.addService(helloProto.Greeter.service, { sayHello: sayHello });
  server.bindAsync('0.0.0.0:50051', grpc.ServerCredentials.createInsecure(), () => {
    console.log('Server running at http://0.0.0.0:50051');
    server.start();
  });
}

main();
```

Client.js

```
const grpc = require('@grpc/grpc-js');
const protoLoader = require('@grpc/proto-loader');

const packageDefinition = protoLoader.loadSync('helloworld.proto', {
  keepCase: true,
  longs: String,
  enums: String,
  defaults: true,
  oneofs: true
});
const helloProto = grpc.loadPackageDefinition(packageDefinition).helloworld;

function main() {
  const client = new helloProto.Greeter('localhost:50051',
    grpc.credentials.createInsecure());
```



```
const user = process.argv[2] || 'world';

client.sayHello({ name: user }, (err, response) => {
  if (err) {
    console.error(err);
  } else {
    console.log('Greeting:', response.message);
  }
});
}

main();
```

Agora, para testar o nosso projeto, abra dois terminais no seu computador. Em um, execute o comando `node server.js`, e no outro, `node client.js`. Caso tudo esteja configurado corretamente, você deve ter a seguinte mensagem no seu terminal:

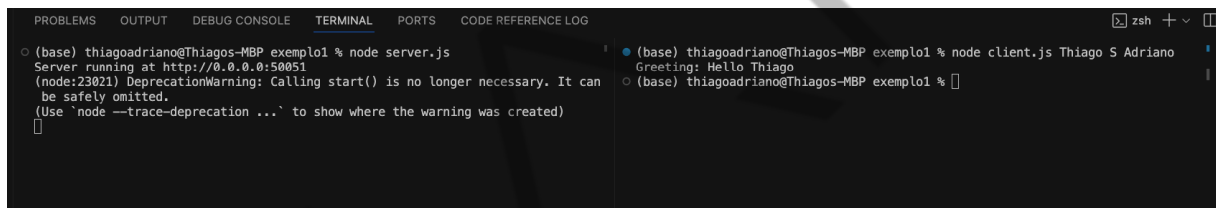


Figura 1 - Exemplo de código rodando com gRPC
Fonte: elaborado pelo autor (2024)

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, aprendemos o básico sobre gRPC, um framework desenvolvido pelo Google. Exploramos como o gRPC é utilizado para comunicação eficiente entre serviços distribuídos. Além disso, tivemos um exemplo de como definir um serviço e as mensagens utilizando protobuf.



REFERÊNCIAS

Introdução ao gRPC com Node.js. **Medium**. 2019. Disponível em: <https://medium.com/xp-inc/introdu%C3%A7%C3%A3o-ao-grpc-com-node-js-98f6a4ede11>. Acesso em: 09 ago. 2024.

O guia completo do gRPC parte 1: O que é gRPC? **Isantos**. 2021. Disponível em: <https://blog.isantos.dev/guia-grpc-1/>. Acesso em: 09 ago. 2024.

Qual é a diferença entre gRPC e REST? **Treinamento WEB**. 2024. Disponível em: <https://aws.amazon.com/pt/compare/the-difference-between-grpc-and-rest/>. Acesso em: 09 ago. 2024.

PALAVRAS-CHAVE

Palavras-chave: gRPC, proto, RPC.

EMSE